CTA Service Cuts Problem Set 6

Peter Ganong, Felix Farb, and Richard Chen

As of September 2025, the Chicago Transit Authority was facing a major budget shortfall which was going to require a 40% cut in service. Here is the Chicago Sun-Times coverage of the issue. Acting President Nora Leerhsen has asked you to teach her about different ways to achieve that service reduction.[1]

She has told you that she cares about both about efficiency, by which she means affecting the smallest number of riders possible, and equity. Although equity has many different definitions, she suggests that using income is sufficient. Here are the datasets you will use:

- Scheduled trips from Chicago Transit Authority in General Transit Feed Specification (GTFS)
- Passengers' actual trips on CTA in Chicago from MIT Transit Lab. These were recently used in the economics paper "Optimal Urban Transportation Policy: Evidence from Chicago''. We are grateful to one of the authors (Juan Camilo Castillo) for sharing it with us. The appendix describes the data in a bit more detail.
- Median Household income by Census tract from the 2023 5 year ACS. Taken from the NHGIS website.

```
# setup
import geopandas as gpd
from os.path import join
import altair as alt
import pandas as pd
import numpy as np
import time

import warnings
warnings.filterwarnings('ignore')
alt.renderers.enable("png")
```

[1] Of course, the same dataset and questions could also be used to think about expanding service in a different budget environment. In this case, by the end of September, after we wrote the problem set, Illinois lawmakers passed a package which covered the CTA funding gap. Details here.

```python
alt.data_transformers.disable_max_rows()

data_path = 'data/external'

# improve graph Resolution
import tempfile
from IPython.display import SVG, display, Image
import vl_convert as vlc

def display_altair_png(chart, scale=2):
    """
    Render an Altair chart to a PNG and display it inline.

    Parameters
    ----------
    chart : altair.Chart
        Altair chart object to render.
    scale : int, optional
        Resolution scaling factor for the PNG (default = 2). Use scale=2 for
↪   standard slides. Use scale = 3-4 for dense figures or PDF exports.
    """
    png_bytes = vlc.vegalite_to_png(chart.to_dict(), scale=scale)

    # Write to a temporary PNG file and display
    with tempfile.NamedTemporaryFile(suffix=".png") as tmp:
        tmp.write(png_bytes)
        tmp.flush()
        display(Image(filename=tmp.name))
```

Data preparation

The functions below clean and merge the CTA schedule, ridership, and income data needed for PS6. For details on the data cleaning steps, see the PS5 solution.

```python
def calculate_trip_counts(verbose=False):
    trips_file = join(data_path, "trips.txt")
    routes_file = join(data_path, "routes.txt")

    trips_df  = pd.read_csv(trips_file, dtype={"route_id": str})
    routes_df = pd.read_csv(routes_file, dtype={"route_id": str})
```

```
    trip_counts = (
        trips_df["route_id"]
        .value_counts()
        .rename_axis("route_id")
        .reset_index(name="trip_count")
    )

    routes_w_trips = routes_df.merge(trip_counts, on="route_id", how="left")
    routes_w_trips["trip_count"] =
 ↪  routes_w_trips["trip_count"].fillna(0).astype(int)

    if verbose:
        routes_in_routes = routes_df["route_id"].nunique()
        routes_in_trips  = trips_df["route_id"].nunique()
        zero_trip_routes = int((routes_w_trips["trip_count"] == 0).sum())
        print(f"  routes.txt: {routes_in_routes} distinct route_id")
        print(f"  trips.txt:  {routes_in_trips} distinct route_id")
        print(f"  Routes with zero trips: {zero_trip_routes}")
    return routes_w_trips


def read_passengers_df():
    trains_file = join(data_path, "train_passengers.csv")
    buses_file = join(data_path, "bus_passengers.csv")
    train_passengers_df = pd.read_csv(trains_file)
    bus_passengers_df = pd.read_csv(buses_file)
    passengers_df = pd.concat([train_passengers_df, bus_passengers_df],
 ↪  ignore_index=True)
    passengers_df.drop(columns=['mode_type'], inplace=True)
    return passengers_df


passengers_df = read_passengers_df()

line_string_rename = {
    'Brown': 'Brn',
    'Green': 'G',
    'Orange': 'Org',
    'Purple': 'P',
    'Yellow': 'Y',
    '5': 'N5',
}
passengers_df['line_string'] =
 ↪  passengers_df['line_string'].replace(line_string_rename)
```

```python
def rider_income_calc(passengers_df):
    tracts_path = join(data_path,
↪   "illinois_tract_income/illinois_tract_income.shp")
    tracts = gpd.read_file(tracts_path)

    psgrs = passengers_df.copy()
    psgrs = psgrs[pd.notna(psgrs['start_lat']) &
↪   pd.notna(psgrs['start_lon'])]

    subtrip_1 = psgrs.loc[psgrs["order"].eq(1)]

    coord_cols = ["start_lat", "start_lon"]
    unique_coords = (
        subtrip_1[coord_cols]
        .drop_duplicates()
        .reset_index(drop=True)
    )

    unique_points = gpd.GeoDataFrame(
        unique_coords,
        geometry=gpd.points_from_xy(unique_coords["start_lon"],
↪   unique_coords["start_lat"]),
        crs="EPSG:4326",
    )

    unique_points = unique_points.to_crs(tracts.crs)
    unique_joined = gpd.sjoin(unique_points, tracts, how="left",
↪   predicate="within")

    if "index_right" in unique_joined.columns:
        unique_joined = unique_joined.drop(columns=["index_right"])

    psgrs_inc = psgrs.merge(
        unique_joined[coord_cols + ["geometry", "GEOID", "ASQPE001"]],
        on=coord_cols,
        how="left",
    )

    out_gdf = gpd.GeoDataFrame(psgrs_inc, geometry="geometry",
↪   crs=tracts.crs)
```

```
    out_gdf["income"] = pd.to_numeric(out_gdf["ASQPE001"], errors="coerce")
    out_gdf = out_gdf[out_gdf["income"] > 0]
    out_gdf = out_gdf.loc[:, ~out_gdf.columns.duplicated()]

    return out_gdf


passengers_inc_df = rider_income_calc(passengers_df)

passengers_inc_df["income_quintile"] = (
    pd.qcut(passengers_inc_df["income"],
     q=5,
     labels=False,
     duplicates="drop").add(1)
    .astype("Int64")
)


def calculate_passenger_counts_quintile(passengers_inc_df):
    total_counts = (
        passengers_inc_df
        .groupby("line_string", as_index=False, sort=False)
        .agg(
            median_income=("income", "median"),
            passenger_count=("line_string", "size")
        )
    )

    group_cols = ["line_string", "income_quintile"]

    by_quintile = (
        passengers_inc_df
        .groupby(group_cols, as_index=False, sort=False)
        .size()
        .rename(columns={"size": "passenger_count"})
    )

    expected_quintiles = range(1, 6)

    by_quintile_wide = (
        by_quintile
        .pivot(index="line_string", columns="income_quintile",
 ↪  values="passenger_count")
```

```
        .reindex(columns=expected_quintiles, fill_value=0)
        .fillna(0)
    )
    by_quintile_wide = by_quintile_wide.astype(int)
    by_quintile_wide.columns = [f"passengers_q{int(c)}" for c in
↪   by_quintile_wide.columns]
    by_quintile_wide = by_quintile_wide.reset_index()

    passenger_count_df = total_counts.merge(by_quintile_wide,
↪   on="line_string", how="left")
    passenger_cols = [f"passengers_q{i}" for i in range(1, 6)]
    passenger_count_df[passenger_cols] =
↪   passenger_count_df[passenger_cols].fillna(0).astype(int)

    return passenger_count_df


def calculate_utilization_rates_quintile(passengers_inc_df):
    passenger_count_df =
↪   calculate_passenger_counts_quintile(passengers_inc_df)
    trips_count_df = calculate_trip_counts()

    utilization_quintile_df = (
        trips_count_df[["route_id", "trip_count", "route_long_name"]]
        .merge(
            passenger_count_df,
            how="inner",
            left_on="route_id",
            right_on="line_string"
        )
    )

    utilization_quintile_df["utilization_rate"] = (
        utilization_quintile_df["passenger_count"] /
↪   utilization_quintile_df["trip_count"]
    )

    for q in range(1, 6):
        col = f"passengers_q{q}"
        utilization_quintile_df[f"utilization_q{q}"] = (
            utilization_quintile_df[col] /
↪   utilization_quintile_df["trip_count"]
```

```
        )

    return utilization_quintile_df


utilization_quintile_df =
 ↪  calculate_utilization_rates_quintile(passengers_inc_df)
print(f"Utilization dataframe: {len(utilization_quintile_df)} routes")
```

PS6 – equity-efficiency tradeoff and dashboards

We prefer that you use your answers to problem 5 as inputs to problem 6. However, even in the event that you are unable to complete problem 5, we have provided the correct final output files from problem 5 to develop your dashboard in problem 6 in the `data/dashboard` folder.

Problem 5

Next, you will create a menu of service cuts for the CTA chief. Each option should come with information about efficiency and equity consequences.

We will make the following simplifying assumptions:

- Service reductions happen across an entire route without making distinctions of the time of day or the day of the week.
- Every trip has the same income distribution as the route as a whole.

For example, suppose that a route has 4 scheduled trips, 40 passengers total, and 8 passengers from each income quintile. If we cut one trip, then 10 passengers will be affected, 2 from each income quintile.

The crucial assumption we will make is that if a trip is cut, then passengers take another train or bus on that route. Once you have cut a trip, then utilization changes on that route because the affected riders are redistributed to other trips. We have given you a function `cut_trips()` which addresses the redistribution issue without any comments at the end of this script.

```
# cut_trips function (provided in code/cut_trips.py)
def cut_trips(df: pd.DataFrame, verbose=False, target_frac: float = 0.60):
    out = df.copy()
    out["trip_count"] = out["trip_count"].astype(int)
    out["passenger_count"] = out["passenger_count"].astype(float)
    total0 = int(out["trip_count"].sum())
    target_total = int(np.floor(target_frac * total0))
    remaining_to_cut = total0 - target_total
    current_trip = out["trip_count"].copy()
    pax = out["passenger_count"]
    iterations = 0
    while remaining_to_cut > 0:
        mask = current_trip > 0
        if not mask.any():
            break
        util = pd.Series(np.inf, index=out.index, dtype=float)
        util.loc[mask] = pax.loc[mask] / current_trip.loc[mask]
```

```
        active_util = util.loc[mask].sort_values(kind="mergesort")
        i = active_util.index[0]
        u_next = active_util.iloc[1] if len(active_util) > 1 else np.inf
        if verbose:
            rid = out.loc[i, "route_id"] if "route_id" in out.columns else i
            print(f"Iter {iterations+1}:
             ↪  remaining_to_cut={remaining_to_cut}")
            print(f"  chosen route={rid}, trips={int(current_trip.loc[i])}, "
                  f"pax={pax.loc[i]}, util={np.round(util.loc[i], 2)}")
            print(f"  u_next={np.round(u_next, 2)}")
        if not np.isfinite(u_next) or u_next <= 0:
            delta = min(remaining_to_cut, int(current_trip.loc[i]))
        else:
            if pax.loc[i] == 0 and u_next > 0:
                delta = min(remaining_to_cut, int(current_trip.loc[i]))
            else:
                threshold = pax.loc[i] / u_next
                delta_min = int(np.ceil(current_trip.loc[i] - threshold))
                delta_min = max(delta_min, 1)
                delta = min(delta_min, int(current_trip.loc[i]),
↪   remaining_to_cut)
        current_trip.loc[i] = int(current_trip.loc[i]) - int(delta)
        remaining_to_cut -= int(delta)
        iterations += 1
        if verbose:
            print(f"  cut={delta}, new_trips={int(current_trip.loc[i])}, "
                  f"remaining={remaining_to_cut}\n")
        iterations += 1
    out["new_trip_count"] = current_trip.astype(int)
    mask_new = out["new_trip_count"] > 0
    out["utilization_rate_new"] = np.where(
        mask_new, out["passenger_count"] / out["new_trip_count"], np.inf
    )
    return out
```

Part a. understand `cut_trips()`

To make things manageable, let's start by focusing on just the routes in a minimum working example. Below are the statistics for three bus routes.

	route_id	median_income	passenger_count	utilization_rate
0	11	103611	175	0.433168
1	68	117583	7997	17.890380
2	57	33516	58	0.111538

Read this function carefully and be sure you understand it. Run it on the `mwe` with the `verbose` parameter set to True to see how the function works in this simple example.

Provide two descriptions of the algorithm:

1. Describe it as you would if you had to explain it to someone who is not a master's student in policy
2. Describe it in pseudocode

Solution

```
mwe_cuts = cut_trips(mwe, verbose=True)
print("\nResult:")
print(mwe_cuts[["route_id", "route_long_name", "trip_count",
                "new_trip_count",
                 ↪  "utilization_rate_new"]].to_string(index=False))
```

```
Iter 1: remaining_to_cut=549
  chosen route=57, trips=520, pax=58.0, util=0.11
  u_next=0.43
  cut=387, new_trips=133, remaining=162

Iter 3: remaining_to_cut=162
  chosen route=11, trips=404, pax=175.0, util=0.43
  u_next=0.44
  cut=3, new_trips=401, remaining=159

Iter 5: remaining_to_cut=159
  chosen route=57, trips=133, pax=58.0, util=0.44
  u_next=0.44
  cut=1, new_trips=132, remaining=158

Iter 7: remaining_to_cut=158
  chosen route=11, trips=401, pax=175.0, util=0.44
  u_next=0.44
  cut=3, new_trips=398, remaining=155
```

```
Iter 9: remaining_to_cut=155
  chosen route=57, trips=132, pax=58.0, util=0.44
  u_next=0.44
  cut=1, new_trips=131, remaining=154

Iter 11: remaining_to_cut=154
  chosen route=11, trips=398, pax=175.0, util=0.44
  u_next=0.44
  cut=3, new_trips=395, remaining=151

Iter 13: remaining_to_cut=151
  chosen route=57, trips=131, pax=58.0, util=0.44
  u_next=0.44
  cut=1, new_trips=130, remaining=150

Iter 15: remaining_to_cut=150
  chosen route=11, trips=395, pax=175.0, util=0.44
  u_next=0.45
  cut=3, new_trips=392, remaining=147

Iter 17: remaining_to_cut=147
  chosen route=57, trips=130, pax=58.0, util=0.45
  u_next=0.45
  cut=1, new_trips=129, remaining=146

Iter 19: remaining_to_cut=146
  chosen route=11, trips=392, pax=175.0, util=0.45
  u_next=0.45
  cut=3, new_trips=389, remaining=143

Iter 21: remaining_to_cut=143
  chosen route=57, trips=129, pax=58.0, util=0.45
  u_next=0.45
  cut=1, new_trips=128, remaining=142

Iter 23: remaining_to_cut=142
  chosen route=11, trips=389, pax=175.0, util=0.45
  u_next=0.45
  cut=3, new_trips=386, remaining=139

Iter 25: remaining_to_cut=139
  chosen route=57, trips=128, pax=58.0, util=0.45
```

```
  u_next=0.45
  cut=1, new_trips=127, remaining=138

Iter 27: remaining_to_cut=138
  chosen route=11, trips=386, pax=175.0, util=0.45
  u_next=0.46
  cut=3, new_trips=383, remaining=135

Iter 29: remaining_to_cut=135
  chosen route=57, trips=127, pax=58.0, util=0.46
  u_next=0.46
  cut=1, new_trips=126, remaining=134

Iter 31: remaining_to_cut=134
  chosen route=11, trips=383, pax=175.0, util=0.46
  u_next=0.46
  cut=3, new_trips=380, remaining=131

Iter 33: remaining_to_cut=131
  chosen route=57, trips=126, pax=58.0, util=0.46
  u_next=0.46
  cut=1, new_trips=125, remaining=130

Iter 35: remaining_to_cut=130
  chosen route=11, trips=380, pax=175.0, util=0.46
  u_next=0.46
  cut=3, new_trips=377, remaining=127

Iter 37: remaining_to_cut=127
  chosen route=57, trips=125, pax=58.0, util=0.46
  u_next=0.46
  cut=1, new_trips=124, remaining=126

Iter 39: remaining_to_cut=126
  chosen route=11, trips=377, pax=175.0, util=0.46
  u_next=0.47
  cut=3, new_trips=374, remaining=123

Iter 41: remaining_to_cut=123
  chosen route=57, trips=124, pax=58.0, util=0.47
  u_next=0.47
  cut=1, new_trips=123, remaining=122
```

```
Iter 43: remaining_to_cut=122
  chosen route=11, trips=374, pax=175.0, util=0.47
  u_next=0.47
  cut=3, new_trips=371, remaining=119

Iter 45: remaining_to_cut=119
  chosen route=57, trips=123, pax=58.0, util=0.47
  u_next=0.47
  cut=1, new_trips=122, remaining=118

Iter 47: remaining_to_cut=118
  chosen route=11, trips=371, pax=175.0, util=0.47
  u_next=0.48
  cut=3, new_trips=368, remaining=115

Iter 49: remaining_to_cut=115
  chosen route=57, trips=122, pax=58.0, util=0.48
  u_next=0.48
  cut=1, new_trips=121, remaining=114

Iter 51: remaining_to_cut=114
  chosen route=11, trips=368, pax=175.0, util=0.48
  u_next=0.48
  cut=3, new_trips=365, remaining=111

Iter 53: remaining_to_cut=111
  chosen route=57, trips=121, pax=58.0, util=0.48
  u_next=0.48
  cut=1, new_trips=120, remaining=110

Iter 55: remaining_to_cut=110
  chosen route=11, trips=365, pax=175.0, util=0.48
  u_next=0.48
  cut=3, new_trips=362, remaining=107

Iter 57: remaining_to_cut=107
  chosen route=57, trips=120, pax=58.0, util=0.48
  u_next=0.48
  cut=1, new_trips=119, remaining=106

Iter 59: remaining_to_cut=106
  chosen route=11, trips=362, pax=175.0, util=0.48
  u_next=0.49
```

```
  cut=3, new_trips=359, remaining=103

Iter 61: remaining_to_cut=103
  chosen route=57, trips=119, pax=58.0, util=0.49
  u_next=0.49
  cut=1, new_trips=118, remaining=102

Iter 63: remaining_to_cut=102
  chosen route=11, trips=359, pax=175.0, util=0.49
  u_next=0.49
  cut=3, new_trips=356, remaining=99

Iter 65: remaining_to_cut=99
  chosen route=57, trips=118, pax=58.0, util=0.49
  u_next=0.49
  cut=1, new_trips=117, remaining=98

Iter 67: remaining_to_cut=98
  chosen route=11, trips=356, pax=175.0, util=0.49
  u_next=0.5
  cut=3, new_trips=353, remaining=95

Iter 69: remaining_to_cut=95
  chosen route=57, trips=117, pax=58.0, util=0.5
  u_next=0.5
  cut=1, new_trips=116, remaining=94

Iter 71: remaining_to_cut=94
  chosen route=11, trips=353, pax=175.0, util=0.5
  u_next=0.5
  cut=3, new_trips=350, remaining=91

Iter 73: remaining_to_cut=91
  chosen route=11, trips=350, pax=175.0, util=0.5
  u_next=0.5
  cut=1, new_trips=349, remaining=90

Iter 75: remaining_to_cut=90
  chosen route=57, trips=116, pax=58.0, util=0.5
  u_next=0.5
  cut=1, new_trips=115, remaining=89

Iter 77: remaining_to_cut=89
```

```
  chosen route=11, trips=349, pax=175.0, util=0.5
  u_next=0.5
  cut=3, new_trips=346, remaining=86

Iter 79: remaining_to_cut=86
  chosen route=57, trips=115, pax=58.0, util=0.5
  u_next=0.51
  cut=1, new_trips=114, remaining=85

Iter 81: remaining_to_cut=85
  chosen route=11, trips=346, pax=175.0, util=0.51
  u_next=0.51
  cut=3, new_trips=343, remaining=82

Iter 83: remaining_to_cut=82
  chosen route=57, trips=114, pax=58.0, util=0.51
  u_next=0.51
  cut=1, new_trips=113, remaining=81

Iter 85: remaining_to_cut=81
  chosen route=11, trips=343, pax=175.0, util=0.51
  u_next=0.51
  cut=3, new_trips=340, remaining=78

Iter 87: remaining_to_cut=78
  chosen route=57, trips=113, pax=58.0, util=0.51
  u_next=0.51
  cut=1, new_trips=112, remaining=77

Iter 89: remaining_to_cut=77
  chosen route=11, trips=340, pax=175.0, util=0.51
  u_next=0.52
  cut=3, new_trips=337, remaining=74

Iter 91: remaining_to_cut=74
  chosen route=57, trips=112, pax=58.0, util=0.52
  u_next=0.52
  cut=1, new_trips=111, remaining=73

Iter 93: remaining_to_cut=73
  chosen route=11, trips=337, pax=175.0, util=0.52
  u_next=0.52
  cut=3, new_trips=334, remaining=70
```

```
Iter 95: remaining_to_cut=70
  chosen route=57, trips=111, pax=58.0, util=0.52
  u_next=0.52
  cut=1, new_trips=110, remaining=69

Iter 97: remaining_to_cut=69
  chosen route=11, trips=334, pax=175.0, util=0.52
  u_next=0.53
  cut=3, new_trips=331, remaining=66

Iter 99: remaining_to_cut=66
  chosen route=57, trips=110, pax=58.0, util=0.53
  u_next=0.53
  cut=1, new_trips=109, remaining=65

Iter 101: remaining_to_cut=65
  chosen route=11, trips=331, pax=175.0, util=0.53
  u_next=0.53
  cut=3, new_trips=328, remaining=62

Iter 103: remaining_to_cut=62
  chosen route=57, trips=109, pax=58.0, util=0.53
  u_next=0.53
  cut=1, new_trips=108, remaining=61

Iter 105: remaining_to_cut=61
  chosen route=11, trips=328, pax=175.0, util=0.53
  u_next=0.54
  cut=3, new_trips=325, remaining=58

Iter 107: remaining_to_cut=58
  chosen route=57, trips=108, pax=58.0, util=0.54
  u_next=0.54
  cut=1, new_trips=107, remaining=57

Iter 109: remaining_to_cut=57
  chosen route=11, trips=325, pax=175.0, util=0.54
  u_next=0.54
  cut=3, new_trips=322, remaining=54

Iter 111: remaining_to_cut=54
  chosen route=57, trips=107, pax=58.0, util=0.54
```

```
  u_next=0.54
  cut=1, new_trips=106, remaining=53

Iter 113: remaining_to_cut=53
  chosen route=11, trips=322, pax=175.0, util=0.54
  u_next=0.55
  cut=3, new_trips=319, remaining=50

Iter 115: remaining_to_cut=50
  chosen route=57, trips=106, pax=58.0, util=0.55
  u_next=0.55
  cut=1, new_trips=105, remaining=49

Iter 117: remaining_to_cut=49
  chosen route=11, trips=319, pax=175.0, util=0.55
  u_next=0.55
  cut=3, new_trips=316, remaining=46

Iter 119: remaining_to_cut=46
  chosen route=57, trips=105, pax=58.0, util=0.55
  u_next=0.55
  cut=1, new_trips=104, remaining=45

Iter 121: remaining_to_cut=45
  chosen route=11, trips=316, pax=175.0, util=0.55
  u_next=0.56
  cut=3, new_trips=313, remaining=42

Iter 123: remaining_to_cut=42
  chosen route=57, trips=104, pax=58.0, util=0.56
  u_next=0.56
  cut=1, new_trips=103, remaining=41

Iter 125: remaining_to_cut=41
  chosen route=11, trips=313, pax=175.0, util=0.56
  u_next=0.56
  cut=3, new_trips=310, remaining=38

Iter 127: remaining_to_cut=38
  chosen route=57, trips=103, pax=58.0, util=0.56
  u_next=0.56
  cut=1, new_trips=102, remaining=37
```

```
Iter 129: remaining_to_cut=37
  chosen route=11, trips=310, pax=175.0, util=0.56
  u_next=0.57
  cut=3, new_trips=307, remaining=34

Iter 131: remaining_to_cut=34
  chosen route=57, trips=102, pax=58.0, util=0.57
  u_next=0.57
  cut=1, new_trips=101, remaining=33

Iter 133: remaining_to_cut=33
  chosen route=11, trips=307, pax=175.0, util=0.57
  u_next=0.57
  cut=3, new_trips=304, remaining=30

Iter 135: remaining_to_cut=30
  chosen route=57, trips=101, pax=58.0, util=0.57
  u_next=0.58
  cut=1, new_trips=100, remaining=29

Iter 137: remaining_to_cut=29
  chosen route=11, trips=304, pax=175.0, util=0.58
  u_next=0.58
  cut=3, new_trips=301, remaining=26

Iter 139: remaining_to_cut=26
  chosen route=57, trips=100, pax=58.0, util=0.58
  u_next=0.58
  cut=1, new_trips=99, remaining=25

Iter 141: remaining_to_cut=25
  chosen route=11, trips=301, pax=175.0, util=0.58
  u_next=0.59
  cut=3, new_trips=298, remaining=22

Iter 143: remaining_to_cut=22
  chosen route=57, trips=99, pax=58.0, util=0.59
  u_next=0.59
  cut=1, new_trips=98, remaining=21

Iter 145: remaining_to_cut=21
  chosen route=11, trips=298, pax=175.0, util=0.59
  u_next=0.59
```

```
  cut=3, new_trips=295, remaining=18

Iter 147: remaining_to_cut=18
  chosen route=57, trips=98, pax=58.0, util=0.59
  u_next=0.59
  cut=1, new_trips=97, remaining=17

Iter 149: remaining_to_cut=17
  chosen route=11, trips=295, pax=175.0, util=0.59
  u_next=0.6
  cut=3, new_trips=292, remaining=14

Iter 151: remaining_to_cut=14
  chosen route=57, trips=97, pax=58.0, util=0.6
  u_next=0.6
  cut=1, new_trips=96, remaining=13

Iter 153: remaining_to_cut=13
  chosen route=11, trips=292, pax=175.0, util=0.6
  u_next=0.6
  cut=3, new_trips=289, remaining=10

Iter 155: remaining_to_cut=10
  chosen route=57, trips=96, pax=58.0, util=0.6
  u_next=0.61
  cut=1, new_trips=95, remaining=9

Iter 157: remaining_to_cut=9
  chosen route=11, trips=289, pax=175.0, util=0.61
  u_next=0.61
  cut=3, new_trips=286, remaining=6

Iter 159: remaining_to_cut=6
  chosen route=57, trips=95, pax=58.0, util=0.61
  u_next=0.61
  cut=1, new_trips=94, remaining=5

Iter 161: remaining_to_cut=5
  chosen route=11, trips=286, pax=175.0, util=0.61
  u_next=0.62
  cut=3, new_trips=283, remaining=2

Iter 163: remaining_to_cut=2
```

```
  chosen route=57, trips=94, pax=58.0, util=0.62
  u_next=0.62
  cut=1, new_trips=93, remaining=1

Iter 165: remaining_to_cut=1
  chosen route=11, trips=283, pax=175.0, util=0.62
  u_next=0.62
  cut=1, new_trips=282, remaining=0


Result:
 route_id   route_long_name  trip_count  new_trip_count  utilization_rate_new
       11           Lincoln         404             282              0.620567
       68 Northwest Highway         447             447             17.890380
       57           Laramie         520              93              0.623656
```

1. Plain English description:

The algorithm reduces the total number of bus and train trips by 40% (keeping 60%). It does this by repeatedly finding the route that is least used (i.e., has the fewest passengers per trip). It then removes trips from that route until its passengers-per-trip ratio rises to match the next-least-used route. At that point, both routes are equally utilized, so on the next round the algorithm may switch to cutting from either one. This process repeats until the target number of trips has been removed. Routes with zero passengers are cut entirely first.

2. Pseudocode:

```
FUNCTION cut_trips(data, target_fraction = 0.60):
    total_trips <- sum of all route trip counts
    target <- floor(target_fraction * total_trips)
    remaining_to_cut <- total_trips - target

    WHILE remaining_to_cut > 0:
        FOR each route with trips > 0:
            compute utilization <- passengers / trips

        Find route i with LOWEST utilization
        Find SECOND-lowest utilization u_next among OTHER routes

        IF u_next is infinity or route i has 0 passengers:
            delta <- min(remaining_to_cut, trips on route i)
        ELSE:
            threshold <- passengers_i / u_next
```

```
            delta <- ceil(trips_i - threshold)
            delta <- max(delta, 1)
            delta <- min(delta, trips_i, remaining_to_cut)

        Reduce trips on route i by delta
        remaining_to_cut <- remaining_to_cut - delta

    Record new_trip_count and new utilization for each route
    RETURN updated data
```

Part b. record equity impacts

We will consider three measures of passenger welfare

1. *'Passengers affected:'* The number of passengers on routes with longer wait time (because there has been at least some service cut). This metric is equal to zero if there are no cuts to a route and equal to the total number of passengers using the route if there are cuts to a route.
2. *'Average wait time:'* The number of minutes in a day divided by the number of trips along that route in a day, which is averaged up weighted by passenger count in order to get average wait times.
3. *'Passengers with lost service:'* The number of passengers on routes with zero trips.

Name the data frame produced by `cut_trips()` as `df_with_cuts`. Write a new function which takes the data frame `df_with_cuts` and returns two data frames.

1. `df_with_cuts` plus new columns with the overall number of affected passengers, average wait times, and the number of passengers with no service, as well as those same metrics broken down by income quintile.
2. `summary_df` which has the total number of affected passengers and the number of passengers with lost service by income quintile.

Run `compute_cut_impacts(cut_trips(mwe))` to confirm the calculations on your minimal working example.

Solution

```python
def compute_cut_impacts(df_with_cuts):
    df = df_with_cuts.copy()

    df["cut_trip_count"] = df["trip_count"] - df["new_trip_count"]
    df["cut_trip_prop"] = df["cut_trip_count"] / df["trip_count"]
```

```
# Affected passengers (binary: all passengers if any cuts, zero
 ↪ otherwise)
has_cut = df["cut_trip_count"] > 0
for q in range(1, 6):
    df[f"affected_q{q}"] = np.where(has_cut, df[f"passengers_q{q}"], 0)
df["affected_total"] = np.where(has_cut, df["passenger_count"], 0)

# Passengers with no service (routes fully eliminated)
df["no_service"] = df["new_trip_count"] == 0
for q in range(1, 6):
    df[f"no_service_q{q}"] = np.where(
        df["no_service"], df[f"passengers_q{q}"], 0
    )
df["no_service_total"] = np.where(
    df["no_service"], df["passenger_count"], 0
)

# Expected wait time: 1440 min/day / trips, capped at 15 for no-service
 ↪ routes
df["expected_wait"] = np.where(
    df["new_trip_count"] > 0,
    1440 * 7 / df["new_trip_count"],
    15.0
)

# Build summary
summary = {}
summary["total_affected"] = df["affected_total"].sum()
for q in range(1, 6):
    summary[f"affected_q{q}"] = df[f"affected_q{q}"].sum()
summary["no_service_total"] = df["no_service_total"].sum()
for q in range(1, 6):
    summary[f"no_service_q{q}"] = df[f"no_service_q{q}"].sum()
summary["avg_wait"] = np.average(df["expected_wait"],
                                 weights=df["passenger_count"])
for q in range(1, 6):
    pcol = f"passengers_q{q}"
    if df[pcol].sum() > 0:
        summary[f"avg_wait_q{q}"] = np.average(
            df["expected_wait"], weights=df[pcol]
        )
    else:
        summary[f"avg_wait_q{q}"] = np.nan
```

```
    summary_df = pd.DataFrame([summary])
    return df, summary_df
```

```
# Test on MWE
mwe_efficiency = cut_trips(mwe)
mwe_eff_detail, mwe_eff_summary = compute_cut_impacts(mwe_efficiency)

print("MWE efficiency cut results:")
print(mwe_eff_detail[["route_id", "route_long_name", "trip_count",
                      "new_trip_count",
                       ↪  "affected_total"]].to_string(index=False))
print("\nSummary:")
print(mwe_eff_summary.T.to_string())
```

```
MWE efficiency cut results:
 route_id   route_long_name  trip_count  new_trip_count  affected_total
       11           Lincoln         404             282           175.0
       68 Northwest Highway         447             447             0.0
       57           Laramie         520              93            58.0

Summary:
                          0
total_affected    233.000000
affected_q1        59.000000
affected_q2        17.000000
affected_q3        20.000000
affected_q4        17.000000
affected_q5       120.000000
no_service_total    0.000000
no_service_q1       0.000000
no_service_q2       0.000000
no_service_q3       0.000000
no_service_q4       0.000000
no_service_q5       0.000000
avg_wait           23.435821
avg_wait_q1        27.044993
avg_wait_q2        23.659030
avg_wait_q3        22.943610
avg_wait_q4        23.202294
avg_wait_q5        22.880814
```

Part c. propose efficiency-focused cut

Using `cut_trips()`, propose a `new_trip_count` for every route which cuts the lowest-utilization routes first until 40% of trips have been cut.

Solution

The `cut_trips()` function naturally implements the efficiency-focused strategy: it iteratively cuts from the route with the lowest utilization rate until 40% of trips have been removed.

```
def run_efficiency_cut(utilization_quintile_df):
    df_with_cuts = cut_trips(utilization_quintile_df)
    df_impacts, summary = compute_cut_impacts(df_with_cuts)
    return df_impacts, summary

df_eff, summary_eff = run_efficiency_cut(utilization_quintile_df)
print("Efficiency cut summary:")
print(f"  Total affected: {summary_eff['total_affected'].values[0]:.0f}")
print(f"  No service total: {summary_eff['no_service_total'].values[0]:.0f}")
print(f"  Avg wait: {summary_eff['avg_wait'].values[0]:.2f}")
```

```
Efficiency cut summary:
  Total affected: 316483
  No service total: 28
  Avg wait: 19.11
```

Part d. propose equity-focused cut

Make a list of trips to cut according to a 'Rawlsian' rule. Rawls proposed maximizing welfare as the well-being of the worst-off citizen. In the context of this problem, we will define that as the well-being of the lowest income quintile. Sort routes by the number of passengers in the route who are in the bottom quintile[2], and cut trips in that order until 40% of trips have been cut.

If you are finding this exercise to be too difficult, you can go to office hours to see on paper a description of pseudo-code that will help with this problem.

Solution

The Rawlsian rule sorts routes by the number of Q1 (lowest income quintile) passengers in ascending order, then cuts all trips from each route sequentially until 40% of total trips have been cut. The last route in the sequence may only be partially cut.

[2] One might also think to order routes by the portion of passengers on the route who are in the bottom quintile.

```python
def cut_trips_low_q1(utilization_quintile_df):
    df = utilization_quintile_df.copy()
    total_trips = int(df["trip_count"].sum())
    target_remaining = int(np.floor(0.60 * total_trips))
    trips_to_cut = total_trips - target_remaining

    # Sort by Q1 passengers ascending (fewest Q1 riders cut first)
    df = df.sort_values("passengers_q1",
↪   ascending=True).reset_index(drop=True)
    df["new_trip_count"] = df["trip_count"].astype(int)

    remaining = trips_to_cut
    for i in range(len(df)):
        if remaining <= 0:
            break
        route_trips = int(df.loc[i, "trip_count"])
        if route_trips <= remaining:
            df.loc[i, "new_trip_count"] = 0
            remaining -= route_trips
        else:
            df.loc[i, "new_trip_count"] = route_trips - remaining
            remaining = 0

    # New utilization
    mask = df["new_trip_count"] > 0
    df["utilization_rate_new"] = np.where(
        mask, df["passenger_count"] / df["new_trip_count"], np.inf
    )
    return df

updated_df_lowq1 = cut_trips_low_q1(utilization_quintile_df)
df_lowq1, summary_lowq1 = compute_cut_impacts(updated_df_lowq1)

print("Rawlsian cut summary:")
print(f"  Total affected: {summary_lowq1['total_affected'].values[0]:.0f}")
print(f"  No service total:
↪   {summary_lowq1['no_service_total'].values[0]:.0f}")
print(f"  Avg wait: {summary_lowq1['avg_wait'].values[0]:.2f}")
```

```
Rawlsian cut summary:
  Total affected: 142094
  No service total: 135823
```

```
Avg wait: 12.89
```

Part e. propose balancing equity and efficiency

In this part, you will create a menu of cuts which prioritizes a balance of equity and efficiency. The idea of trading off efficiency and equity considerations should be familiar to you from micro I in the core, when you looked at maximizing different social welfare functions and explored how different social welfare functions would lead to different policy allocations. In this problem we will choose one specific way to trade off efficiency and equity which is that we will create a 'score'.

The score is utilization rate divided by median rider income.

Order routes according to this score, and cut trips from routes with the lowest score until 40% of trips have been cut. Explain why cutting routes with the lowest score is implementing an equity-efficiency tradeoff.

If you are finding this exercise to be too difficult, you can go to office hours to see on paper a description of pseudo-code that will help with this problem.

Solution

The equity-minded score is `utilization_rate / median_income`. A low score indicates a route that is both underutilized (efficient to cut) and serves higher-income riders (equitable to cut). By cutting routes with the lowest score first, we preferentially cut routes that are underused *and* serve wealthier riders, thereby protecting both overall ridership and lower-income communities.

```
def cut_trips_equity_minded(utilization_quintile_df):
    df = utilization_quintile_df.copy()
    df["equity_score"] = df["utilization_rate"] / df["median_income"]

    total_trips = int(df["trip_count"].sum())
    target_remaining = int(np.floor(0.60 * total_trips))
    trips_to_cut = total_trips - target_remaining

    # Sort by equity score ascending (lowest score cut first)
    df = df.sort_values("equity_score",
↪   ascending=True).reset_index(drop=True)
    df["new_trip_count"] = df["trip_count"].astype(int)

    remaining = trips_to_cut
    for i in range(len(df)):
        if remaining <= 0:
```

```
            break
        route_trips = int(df.loc[i, "trip_count"])
        if route_trips <= remaining:
            df.loc[i, "new_trip_count"] = 0
            remaining -= route_trips
        else:
            df.loc[i, "new_trip_count"] = route_trips - remaining
            remaining = 0

    mask = df["new_trip_count"] > 0
    df["utilization_rate_new"] = np.where(
        mask, df["passenger_count"] / df["new_trip_count"], np.inf
    )
    return df

updated_df_equityminded = cut_trips_equity_minded(utilization_quintile_df)
df_equity, summary_equity = compute_cut_impacts(updated_df_equityminded)

print("Equity-minded cut summary:")
print(f"  Total affected: {summary_equity['total_affected'].values[0]:.0f}")
print(f"  No service total:
 ↪  {summary_equity['no_service_total'].values[0]:.0f}")
print(f"  Avg wait: {summary_equity['avg_wait'].values[0]:.2f}")
```

```
Equity-minded cut summary:
  Total affected: 103955
  No service total: 99687
  Avg wait: 13.76
```

Part f. summarize impacts of each approach

Coding

1. Using the full sample with all the routes, make a table comparing the number of passengers affected by income quintile under each method. What do you notice?
2. Make the same table two more times, but using metrics of number of passengers with no service, as well as average wait times.

Diagnostic: first run your code on the minimum working example. Below are the results which you should expect for that example.

Solution

Method	Total passengers affected	Passengers affected by quintile Q1	Q2	Q3	Q4	Q5
40% cut by lowest utilization	233	59	17	20	17	120
40% cut by lowest Q1 passengers	233	59	17	20	17	120
40% cut by equity minded	233	59	17	20	17	120

Method	Total passengers without service	Passengers without service by quintile Q1	Q2	Q3	Q4	Q5
40% cut by lowest utilization	0	0	0	0	0	0
40% cut by lowest Q1 passengers	175	17	8	20	10	120
40% cut by equity minded	58	42	9	0	7	0

Method	Average wait time (Δ vs baseline)	Average wait time by quintile (Δ vs baseline) Q1	Q2	Q3	Q4	Q5
Baseline	22.58	22.44	22.54	22.62	22.55	22.61
40% cut by lowest utilization	23.44	27.04	23.66	22.94	23.20	22.88
	(0.86)	(4.60)	(1.12)	(0.32)	(0.65)	(0.27)
40% cut by lowest Q1 passengers	22.42	22.61	22.52	22.33	22.51	22.36
	(-0.16)	(0.17)	(-0.02)	(-0.30)	(-0.04)	(-0.25)
40% cut by equity minded	22.59	22.26	22.51	22.68	22.54	22.66
	(0.01)	(-0.18)	(-0.03)	(0.06)	(-0.01)	(0.05)

```python
# Run all three methods on MWE
mwe_eff_cuts = cut_trips(mwe)
_, mwe_s_eff = compute_cut_impacts(mwe_eff_cuts)

mwe_lowq1_cuts = cut_trips_low_q1(mwe)
_, mwe_s_lowq1 = compute_cut_impacts(mwe_lowq1_cuts)

mwe_eq_cuts = cut_trips_equity_minded(mwe)
_, mwe_s_eq = compute_cut_impacts(mwe_eq_cuts)

# Baseline wait times
mwe_baseline = mwe.copy()
mwe_baseline["new_trip_count"] = mwe_baseline["trip_count"]
_, mwe_s_base = compute_cut_impacts(mwe_baseline)

methods = ["Baseline",
           "40% cut by lowest utilization",
           "40% cut by lowest Q1 passengers",
           "40% cut by equity minded"]
summaries_mwe = [mwe_s_base, mwe_s_eff, mwe_s_lowq1, mwe_s_eq]
```

```python
# Table 1: Passengers affected (MWE)
print("=" * 70)
print("TABLE 1: Passengers Affected (MWE)")
print("=" * 70)
header = f"{'Method':<35} {'Total':>7} {'Q1':>6} {'Q2':>6} {'Q3':>6}
 ↪  {'Q4':>6} {'Q5':>6}"
print(header)
print("-" * 70)
for name, s in zip(methods[1:], summaries_mwe[1:]):
    row = (f"{name:<35} "
           f"{s['total_affected'].values[0]:>7.0f} "
           f"{s['affected_q1'].values[0]:>6.0f} "
           f"{s['affected_q2'].values[0]:>6.0f} "
           f"{s['affected_q3'].values[0]:>6.0f} "
           f"{s['affected_q4'].values[0]:>6.0f} "
           f"{s['affected_q5'].values[0]:>6.0f}")
    print(row)
```

```
======================================================================
TABLE 1: Passengers Affected (MWE)
======================================================================
```

```
Method                                 Total     Q1     Q2     Q3     Q4     Q5
----------------------------------------------------------------------
40% cut by lowest utilization            233     59     17     20     17    120
40% cut by lowest Q1 passengers          233     59     17     20     17    120
40% cut by equity minded                 233     59     17     20     17    120
```

```python
# Table 2: Passengers with no service (MWE)
print("=" * 70)
print("TABLE 2: Passengers with No Service (MWE)")
print("=" * 70)
print(header)
print("-" * 70)
for name, s in zip(methods[1:], summaries_mwe[1:]):
    row = (f"{name:<35} "
           f"{s['no_service_total'].values[0]:>7.0f} "
           f"{s['no_service_q1'].values[0]:>6.0f} "
           f"{s['no_service_q2'].values[0]:>6.0f} "
           f"{s['no_service_q3'].values[0]:>6.0f} "
           f"{s['no_service_q4'].values[0]:>6.0f} "
           f"{s['no_service_q5'].values[0]:>6.0f}")
    print(row)
```

```
======================================================================
TABLE 2: Passengers with No Service (MWE)
======================================================================
Method                                 Total     Q1     Q2     Q3     Q4     Q5
----------------------------------------------------------------------
40% cut by lowest utilization              0      0      0      0      0      0
40% cut by lowest Q1 passengers          175     17      8     20     10    120
40% cut by equity minded                  58     42      9      0      7      0
```

```python
# Table 3: Average wait time (MWE)
print("=" * 70)
print("TABLE 3: Average Wait Time (MWE)")
print("=" * 70)
wait_header = f"{'Method':<35} {'Avg':>7} {'Q1':>6} {'Q2':>6} {'Q3':>6}
 ↪  {'Q4':>6} {'Q5':>6}"
print(wait_header)
print("-" * 70)

base_wait = mwe_s_base['avg_wait'].values[0]
```

```
base_waits = [mwe_s_base[f'avg_wait_q{q}'].values[0] for q in range(1, 6)]

row = (f"{'Baseline':<35} "
       f"{base_wait:>7.2f} "
       + " ".join(f"{w:>6.2f}" for w in base_waits))
print(row)
print("-" * 70)

for name, s in zip(methods[1:], summaries_mwe[1:]):
    w = s['avg_wait'].values[0]
    wqs = [s[f'avg_wait_q{q}'].values[0] for q in range(1, 6)]
    row = (f"{name:<35} "
           f"{w:>7.2f} "
           + " ".join(f"{wq:>6.2f}" for wq in wqs))
    print(row)
    delta_row = (f"{'(delta)':<35} "
                 f"{w - base_wait:>7.2f} "
                 + " ".join(f"{wq - bw:>6.2f}" for wq, bw in zip(wqs,
                  ↪  base_waits)))
    print(delta_row)
```

```
======================================================================
TABLE 3: Average Wait Time (MWE)
======================================================================
Method                                  Avg     Q1     Q2     Q3     Q4     Q5
----------------------------------------------------------------------
Baseline                              22.58  22.44  22.54  22.62  22.55  22.61
----------------------------------------------------------------------
40% cut by lowest utilization         23.44  27.04  23.66  22.94  23.20  22.88
(delta)                                0.86   4.60   1.12   0.32   0.65   0.27
40% cut by lowest Q1 passengers       22.42  22.61  22.52  22.33  22.51  22.36
(delta)                               -0.16   0.17  -0.02  -0.30  -0.04  -
0.25
40% cut by equity minded              22.59  22.26  22.51  22.68  22.54  22.66
(delta)                                0.01  -0.18  -0.03   0.06  -0.01   0.05
```

Full sample tables:

```
# Baseline for full sample
baseline_df = utilization_quintile_df.copy()
baseline_df["new_trip_count"] = baseline_df["trip_count"]
```

```
_, summary_base = compute_cut_impacts(baseline_df)

methods_full = ["Baseline",
                "40% cut by lowest utilization",
                "40% cut by lowest Q1 passengers",
                "40% cut by equity minded"]
summaries_full = [summary_base, summary_eff, summary_lowq1, summary_equity]
```

```
# Table 1: Passengers affected (full sample)
print("=" * 75)
print("TABLE 1: Passengers Affected (Full Sample)")
print("=" * 75)
header_f = f"{'Method':<35} {'Total':>8} {'Q1':>7} {'Q2':>7} {'Q3':>7}
 ↪  {'Q4':>7} {'Q5':>7}"
print(header_f)
print("-" * 75)
for name, s in zip(methods_full[1:], summaries_full[1:]):
    row = (f"{name:<35} "
           f"{s['total_affected'].values[0]:>8.0f} "
           f"{s['affected_q1'].values[0]:>7.0f} "
           f"{s['affected_q2'].values[0]:>7.0f} "
           f"{s['affected_q3'].values[0]:>7.0f} "
           f"{s['affected_q4'].values[0]:>7.0f} "
           f"{s['affected_q5'].values[0]:>7.0f}")
    print(row)
```

```
===========================================================================
TABLE 1: Passengers Affected (Full Sample)
===========================================================================
Method                                  Total      Q1      Q2      Q3      Q4      Q5
---------------------------------------------------------------------------
40% cut by lowest utilization          316483   71204   48221   52514   68477   76067
40% cut by lowest Q1 passengers        142094   17306   18845   31131   28634   46178
40% cut by equity minded               103955   16586   15136   18816   20045   33372
```

```
# Table 2: Passengers with no service (full sample)
print("=" * 75)
print("TABLE 2: Passengers with No Service (Full Sample)")
print("=" * 75)
print(header_f)
print("-" * 75)
```

```
for name, s in zip(methods_full[1:], summaries_full[1:]):
    row = (f"{name:<35} "
           f"{s['no_service_total'].values[0]:>8.0f} "
           f"{s['no_service_q1'].values[0]:>7.0f} "
           f"{s['no_service_q2'].values[0]:>7.0f} "
           f"{s['no_service_q3'].values[0]:>7.0f} "
           f"{s['no_service_q4'].values[0]:>7.0f} "
           f"{s['no_service_q5'].values[0]:>7.0f}")
    print(row)
```

```
============================================================================
TABLE 2: Passengers with No Service (Full Sample)
============================================================================
Method                                  Total      Q1      Q2      Q3      Q4      Q5
----------------------------------------------------------------------------
40% cut by lowest utilization              28      13       4       5       2       4
40% cut by lowest Q1 passengers        135823   16468   18311   29955   27778   43311
40% cut by equity minded                99687   15702   14238   18336   19251   32160
```

```
# Table 3: Average wait time (full sample)
print("=" * 75)
print("TABLE 3: Average Wait Time (Full Sample)")
print("=" * 75)
print(header_f)
print("-" * 75)

bw = summary_base['avg_wait'].values[0]
bwqs = [summary_base[f'avg_wait_q{q}'].values[0] for q in range(1, 6)]
row = (f"{'Baseline':<35} "
       f"{bw:>8.2f} "
       + " ".join(f"{w:>7.2f}" for w in bwqs))
print(row)
print("-" * 75)

for name, s in zip(methods_full[1:], summaries_full[1:]):
    w = s['avg_wait'].values[0]
    wqs = [s[f'avg_wait_q{q}'].values[0] for q in range(1, 6)]
    row = (f"{name:<35} "
           f"{w:>8.2f} "
           + " ".join(f"{wq:>7.2f}" for wq in wqs))
    print(row)
```

```
    delta_row = (f"{'  (delta)':<35} "
                 f"{w - bw:>8.2f} "
                 + " ".join(f"{wq - b:>7.2f}" for wq, b in zip(wqs, bwqs)))
    print(delta_row)
```

```
==========================================================================
TABLE 3: Average Wait Time (Full Sample)
==========================================================================
Method                                  Total      Q1      Q2      Q3      Q4      Q5
---------------------------------------------------------------------------
Baseline                                13.52   13.50   13.61   14.50   13.19   12.79
---------------------------------------------------------------------------
40% cut by lowest utilization           19.11   19.34   17.77   19.77   18.71   19.97
  (delta)                                5.60    5.84    4.16    5.26    5.52    7.18
40% cut by lowest Q1 passengers         12.89   12.84   13.23   13.46   12.40   12.57
  (delta)                               -0.62   -0.66   -0.38   -1.04   -
0.79   -0.22
40% cut by equity minded                13.76   13.62   13.81   14.76   13.31   13.34
  (delta)                                0.24    0.12    0.20    0.25    0.12    0.55
```

Thinking

3. What are the advantages of the three different metrics in assessing the policies? Discuss the limitations of the assumptions that we have made, both stated and unstated.
4. Using the tables, analyze the difference between the policies.
5. Discuss the effectiveness of the 'Rawlsian' rule.

In considering how to answer these questions, here is a small concrete example of how to read the MWE tables:

Notice that Table 1 shows identical affected-passenger counts (233) across all three methods on the MWE. This is because the binary "affected" metric counts all passengers on any route with *any* cut, and on this small 3-route example the same 2 routes always get cut. Table 1 therefore cannot distinguish between methods on the MWE — but Tables 2 and 3 can. In Table 2, the efficiency method eliminates zero routes entirely, while the Rawlsian method leaves 175 passengers with no service. In Table 3, the efficiency method increases Q1 wait times by 4.60 minutes, far more than the other methods. This kind of cross-table reasoning — using no-service and wait-time tables to see differences that the affected-passengers table cannot reveal — is essential when analyzing the full sample results.

```python
corr = mwe["median_income"].corr(mwe["utilization_rate"])
print("Correlation between income and utilization rate:", corr)
```

```
Correlation between income and utilization rate: 0.6404743594290567
```

Solution

3. Advantages and limitations of the three metrics:

- *Passengers affected* is simple and measures total disruption breadth. Because it uses a binary definition (all passengers count as affected if there is any cut), it treats a single-trip reduction the same as near-total elimination. This makes it a blunt instrument that overstates impact for minor reductions.
- *Average wait time* captures the intensity of disruption – longer waits matter more. It reflects the actual experience of riders. However, it treats all passengers equally regardless of alternatives available.
- *Passengers with lost service* captures the most extreme outcome – complete elimination of a route. This is the most consequential impact since those riders lose all access.

Limitations of assumptions:

- Service cuts are uniform across the route (no time-of-day or day-of-week variation)
- Every trip has the same income distribution as the route overall
- Affected passengers redistribute to remaining trips on the same route (no switching to other routes or modes)
- Income is proxied by census tract median, not individual income
- The utilization rate is relative (unknown number of days in the data)
- We don't account for geographic accessibility or availability of alternative transit

4. Policy comparison:

- The *efficiency* approach partially cuts many routes (reducing but not eliminating service). Under the binary affected metric, this means a large number of total passengers are counted as affected — every passenger on any route with even one trip removed. However, few routes lose service entirely, and wait time increases are concentrated on low-income riders (visible in the large Q1 wait time increase).
- The *Rawlsian* approach protects Q1 riders by cutting routes with fewest Q1 passengers first. It fully eliminates some routes, resulting in many passengers losing service entirely, but the routes eliminated tend to serve fewer Q1 riders.
- The *equity-minded* approach balances by using the score (utilization/income). Routes with low utilization AND high income are cut first. This spreads impacts more evenly across income groups while keeping total disruption moderate.

5. Effectiveness of the Rawlsian rule:

The Rawlsian rule is effective at its stated goal: it minimizes impacts on Q1 (lowest income) riders by preserving routes that serve many Q1 passengers. However, it comes at a significant cost:

- It leads to a high number of passengers with completely lost service
- It can cut highly-utilized routes simply because they serve few Q1 riders (e.g., express routes through affluent areas), causing large overall disruption
- The "lowest Q1 passengers" ordering doesn't account for utilization, so it may eliminate routes that serve many riders from other quintiles

A more nuanced Rawlsian approach might order by the *proportion* of Q1 riders rather than the absolute count, or might combine Q1 protection with utilization thresholds.

Problem 6 streamlit

To summarize your proposals, you will now create a dashboard using the Streamlit package to display key information in an organized and interactive way.

Part a.

First, visualize each of your three proposals.

1. Using the data from `trips_post_cut_efficiency.csv`, plot new utilization rates against median income using your plotting function from Problem 4. More specifically, modify the plotting function to color routes in one of three colors depending on whether their service is unchanged, reduced, or cut entirely.
2. Repeat this plot again for part 5d using `trips_post_cut_equityminded.csv`
3. Repeat this plot again for part 5e using `trips_post_cut_lowq1.csv`

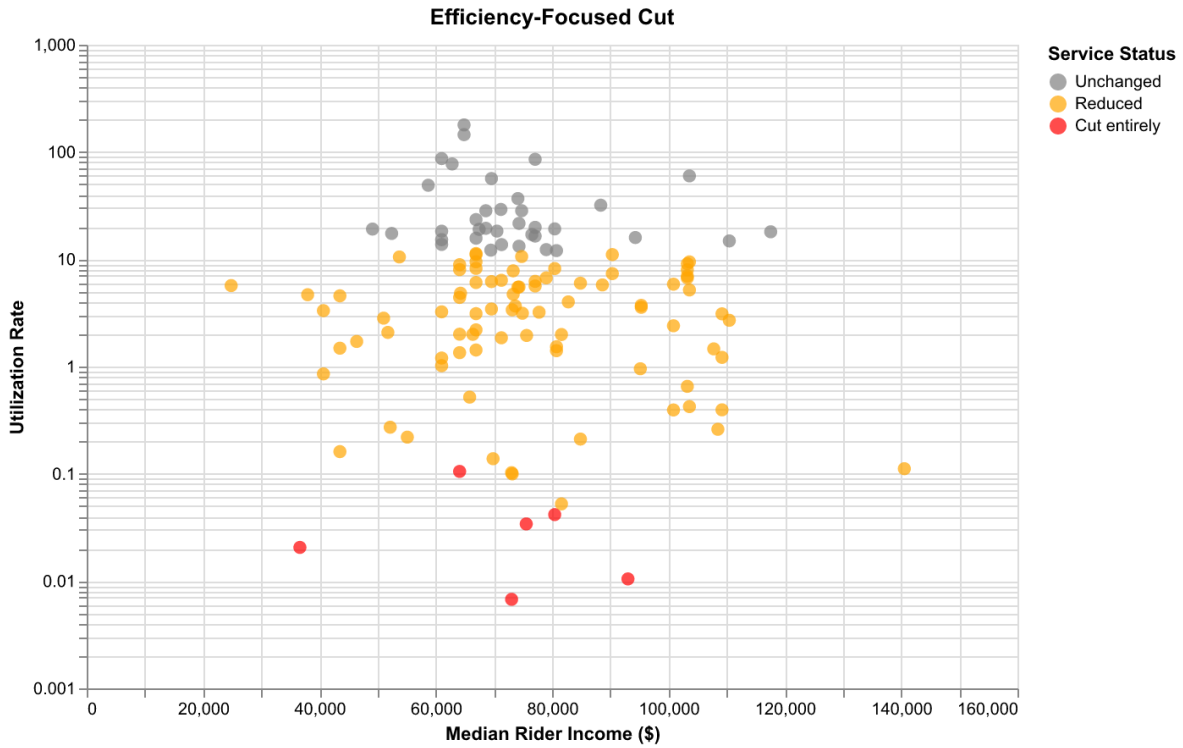
Solution

```
def plot_utilization_vs_income_6a(
    df: pd.DataFrame,
    title_name: str,
    income_col: str = "median_income",
    util_col: str = "utilization_rate",
    filename: str = "utilization_plot.html",
    color_by_change: bool = False,
):
    df = df.copy()
```

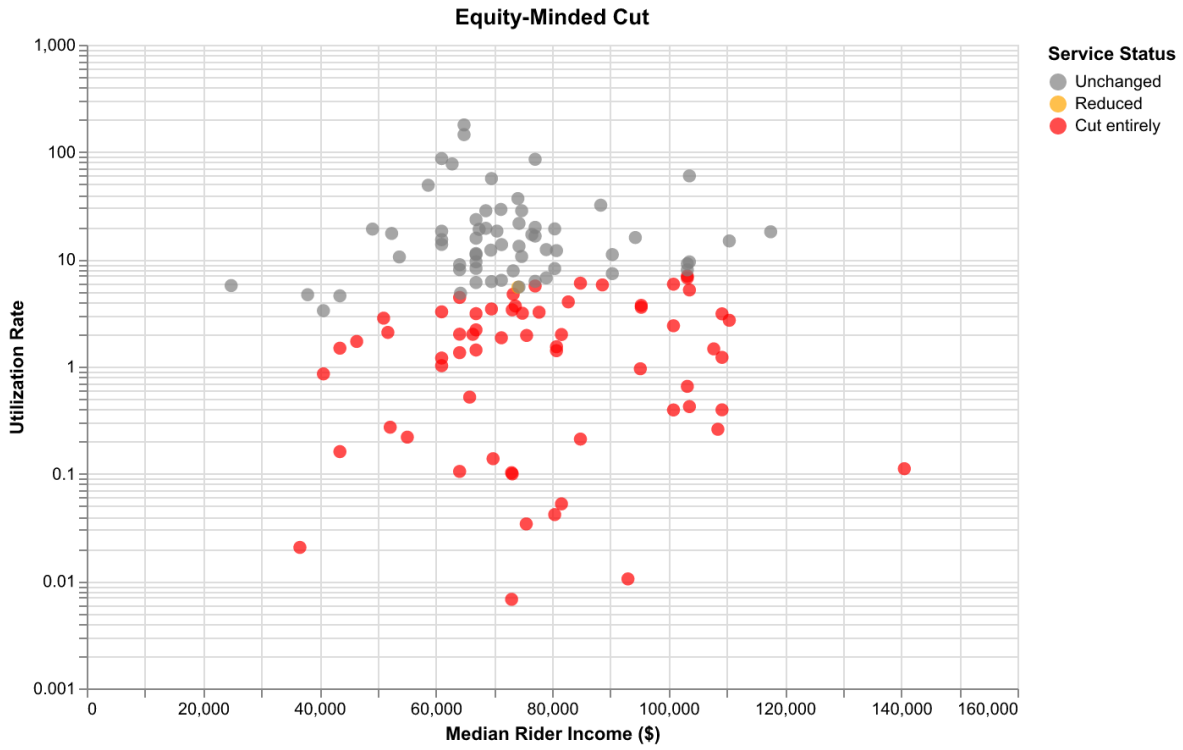
```
    if color_by_change:
        df["change_status"] = np.where(
            df["new_trip_count"] == df["trip_count"], "Unchanged",
            np.where(df["new_trip_count"] == 0, "Cut entirely", "Reduced")
        )
        color_enc = alt.Color(
            "change_status:N",
            scale=alt.Scale(
                domain=["Unchanged", "Reduced", "Cut entirely"],
                range=["gray", "orange", "red"]
            ),
            title="Service Status"
        )
    else:
        color_enc = alt.value("steelblue")

    chart = (
        alt.Chart(df)
        .mark_circle(size=60)
        .encode(
            x=alt.X(f"{income_col}:Q", title="Median Rider Income ($)"),
            y=alt.Y(f"{util_col}:Q", title="Utilization Rate",
                    scale=alt.Scale(type="log")),
            color=color_enc,
            tooltip=["route_long_name:N", f"{income_col}:Q",
                     f"{util_col}:Q", "trip_count:Q", "new_trip_count:Q"],
        )
        .properties(width=550, height=380, title=title_name)
    )
    return chart
```

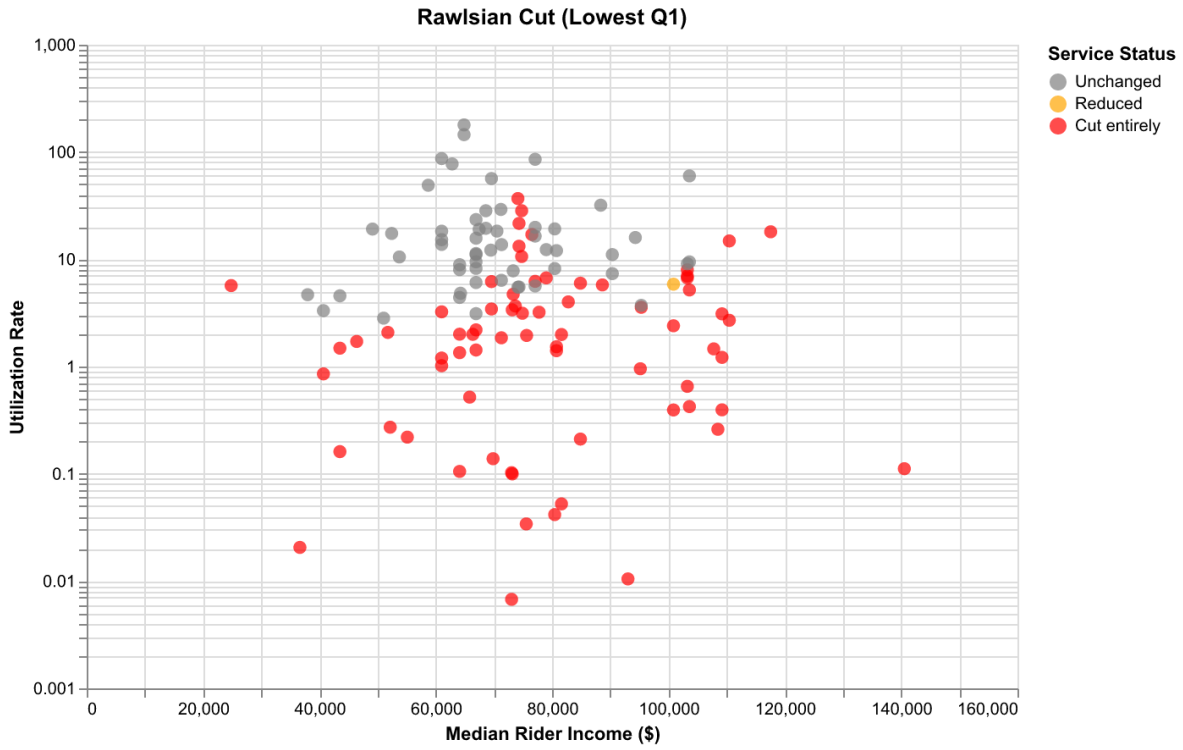
```
# Plot 1: Efficiency
chart_eff = plot_utilization_vs_income_6a(
    df_eff, "Efficiency-Focused Cut", color_by_change=True
)
display_altair_png(chart_eff)
```




```
# Plot 2: Equity-minded
chart_eq = plot_utilization_vs_income_6a(
    df_equity, "Equity-Minded Cut", color_by_change=True
)
display_altair_png(chart_eq)
```




```
# Plot 3: Rawlsian
chart_lq1 = plot_utilization_vs_income_6a(
    df_lowq1, "Rawlsian Cut (Lowest Q1)", color_by_change=True
)
display_altair_png(chart_lq1)
```



Part b.

These static visualizations are helpful, but the major advantage of dashboards are in the interactivity they provide to the user, which allows for a more compact and effectively communicated display of information.

Make a dashboard in streamlit where the user can choose between different policy proposals, whether to show routes with no cuts at all, and whether to show bus, L, or both.

Three components:

1. (easiest) render the plot you made in part (a)
2. (easy) render a table where each row is a route and you provide other helpful columns of information
3. a map that displays the CTA Bus and L lines. Use `pydeck` with the included GEOJSON files for Bus and L routes. Be sure to join the routes data with the trips data correctly. Color each line colored by the number of cut trips on each route, with line thickness determined by the number of affected passengers on each line (using data from the `cut_trips_detail` csv files).

For both the plot and the map, there should be a tooltip giving relevant information on highlighted points/routes.

Solution

Save the following as `app.py` in a folder (e.g., `dashboard/`) and run with `streamlit run dashboard/app.py`.

```
import streamlit as st
import pandas as pd
import numpy as np
import altair as alt
import pydeck as pdk
import json

#   File mappings
DASHBOARD_DIR = "data/dashboard"
EXTERNAL_DIR = "data/external/dashboard"

TRIPS_FILES = {
    "Efficiency": f"{DASHBOARD_DIR}/cut_trips_detail.csv",
    "Equity-minded": f"{DASHBOARD_DIR}/cut_trips_detail_equityminded.csv",
    "Rawlsian": f"{DASHBOARD_DIR}/cut_trips_detail_lowq1.csv",
}

FIG_FILES = {
    "Efficiency": f"{DASHBOARD_DIR}/trips_post_cut_efficiency.csv",
    "Equity-minded": f"{DASHBOARD_DIR}/trips_post_cut_equityminded.csv",
    "Rawlsian": f"{DASHBOARD_DIR}/trips_post_cut_lowq1.csv",
}

RAIL_GEOJSON = f"{EXTERNAL_DIR}/CTA_-_'L'_(Rail)_Lines_20250924.geojson"
BUS_GEOJSON = f"{EXTERNAL_DIR}/CTA_-_Bus_Routes_20250924.geojson"

#   Page config
st.set_page_config(page_title="CTA Service Cut Dashboard", layout="wide")
st.title("CTA Service Cut Dashboard")

#   Sidebar controls
policy = st.sidebar.selectbox("Policy Proposal", list(TRIPS_FILES.keys()))
show_unchanged = st.sidebar.checkbox("Show routes with no cuts", value=True)
mode_filter = st.sidebar.multiselect("Mode", ["Bus", "L"], default=["Bus",
 ↪  "L"])

#   Load data
@st.cache_data
def load_data(policy_name):
```

```
    trips_df = pd.read_csv(TRIPS_FILES[policy_name])
    fig_df = pd.read_csv(FIG_FILES[policy_name])

    # Classify change status
    fig_df["change_status"] = np.where(
        fig_df["new_trip_count"] == fig_df["trip_count"], "Unchanged",
        np.where(fig_df["new_trip_count"] == 0, "Cut entirely", "Reduced")
    )
    trips_df["change_status"] = np.where(
        trips_df["new_trip_count"] == trips_df["trip_count"], "Unchanged",
        np.where(trips_df["new_trip_count"] == 0, "Cut entirely", "Reduced")
    )
    return trips_df, fig_df

trips_df, fig_df = load_data(policy)

#   Filter
if not show_unchanged:
    fig_df = fig_df[fig_df["change_status"] != "Unchanged"]
    trips_df = trips_df[trips_df["change_status"] != "Unchanged"]

#   Column 1: Plot
col1, col2 = st.columns(2)

with col1:
    st.subheader(f"Utilization vs Income ({policy})")

    color_scale = alt.Scale(
        domain=["Unchanged", "Reduced", "Cut entirely"],
        range=["gray", "orange", "red"]
    )

    chart = (
        alt.Chart(fig_df)
        .mark_circle(size=60)
        .encode(
            x=alt.X("median_income:Q", title="Median Rider Income ($)"),
            y=alt.Y("utilization_rate:Q", title="Utilization Rate",
                    scale=alt.Scale(type="log")),
            color=alt.Color("change_status:N", scale=color_scale,
                            title="Service Status"),
            tooltip=["route_long_name:N", "median_income:Q",
                     "utilization_rate:Q", "trip_count:Q",
                     ↪  "new_trip_count:Q"],
```

```
        )
        .properties(width=500, height=400)
    )
    st.altair_chart(chart, use_container_width=True)

#   Column 2: Table
with col2:
    st.subheader("Route Details")
    display_cols = ["route_long_name", "route_id", "trip_count",
                    "new_trip_count", "passenger_count", "median_income",
                    "utilization_rate", "change_status"]
    available_cols = [c for c in display_cols if c in trips_df.columns]
    st.dataframe(
        trips_df[available_cols].sort_values("route_long_name"),
        use_container_width=True, height=400
    )

#   Map
st.subheader(f"CTA Route Map ({policy})")

@st.cache_data
def load_geojson(path):
    with open(path) as f:
        return json.load(f)

def prepare_map_data(geojson, trips_df, route_id_key):
    """Merge trip cut data into geojson features for pydeck."""
    features = []
    for feat in geojson["features"]:
        props = feat.get("properties", {})
        rid = str(props.get(route_id_key, ""))

        match = trips_df[trips_df["route_id"].astype(str) == rid]
        if len(match) > 0:
            row = match.iloc[0]
            cut_count = int(row.get("cut_trip_count", 0))
            affected = float(row.get("affected_total", 0))
        else:
            cut_count = 0
            affected = 0

        coords = feat.get("geometry", {}).get("coordinates", [])
        if feat["geometry"]["type"] == "MultiLineString":
```

```
            for line in coords:
                features.append({
                    "path": line,
                    "name": props.get("ROUTE_ID", rid),
                    "cut_trips": cut_count,
                    "affected": affected,
                    "width": max(1, min(affected / 100, 20)),
                    "color": _cut_color(cut_count),
                })
        elif feat["geometry"]["type"] == "LineString":
            features.append({
                "path": coords,
                "name": props.get("ROUTE_ID", rid),
                "cut_trips": cut_count,
                "affected": affected,
                "width": max(1, min(affected / 100, 20)),
                "color": _cut_color(cut_count),
            })
    return features

def _cut_color(cut_count):
    if cut_count == 0:
        return [150, 150, 150, 120]
    elif cut_count < 100:
        return [255, 165, 0, 200]
    else:
        return [255, 0, 0, 200]

map_features = []
if "Bus" in mode_filter:
    bus_geo = load_geojson(BUS_GEOJSON)
    map_features.extend(prepare_map_data(bus_geo, trips_df, "ROUTE"))
if "L" in mode_filter:
    rail_geo = load_geojson(RAIL_GEOJSON)
    map_features.extend(prepare_map_data(rail_geo, trips_df, "ROUTE_ID"))

if map_features:
    layer = pdk.Layer(
        "PathLayer",
        data=map_features,
        get_path="path",
        get_color="color",
        get_width="width",
```

```
        width_min_pixels=1,
        pickable=True,
    )

    view = pdk.ViewState(
        latitude=41.88, longitude=-87.65, zoom=10, pitch=0
    )

    deck = pdk.Deck(
        layers=[layer],
        initial_view_state=view,
        tooltip={"text": "{name}\nCut trips: {cut_trips}\nAffected:
↪   {affected}"},
    )

    st.pydeck_chart(deck)
else:
    st.info("Select at least one mode (Bus or L) to display the map.")
```